

ARIN5204: Reinforcement Learning

3.2 Actor Critic

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE

The Hong Kong University of Science and Technology (HKUST)

April 20, 2026



- 1 Actor-Critic
- 2 Advantage Actor-Critic (A2C)
- 3 Design Decisions

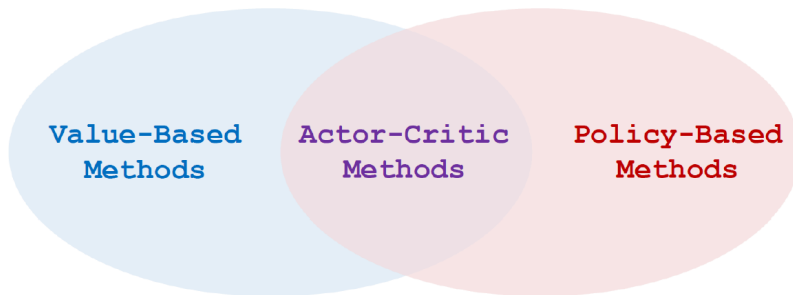
Some slides are from: Sergey Levine (UCB), Shusen Wang (Meta).

① Actor-Critic

② Advantage Actor-Critic (A2C)

③ Design Decisions

Actor-Critic Methods

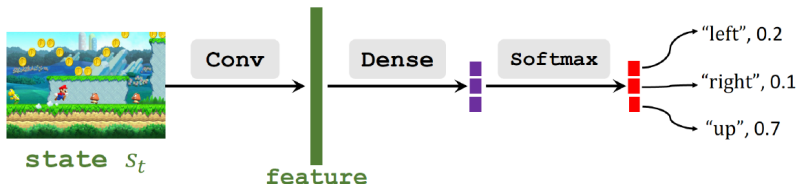


Value Network and Policy Network

- Recall: State-value Function: $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a)$
- Policy network (actor):
 - Use neural net $\pi(a|s; \theta)$ to approximate $\pi(a|s)$
 - θ : trainable parameters of the neural network.
- Value network (critic):
 - Use neural net $Q(s, a; w)$ to approximate $Q_{\pi}(s, a)$
 - w : trainable parameters of the neural network

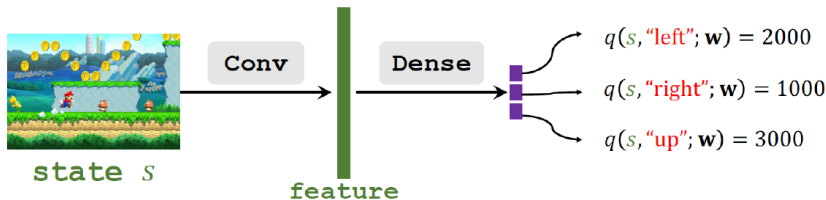
Policy Network (Actor): $\pi(a|s;\theta)$

- **Input:** state s , e.g., a screenshot of Super Mario
- **Output:** probability distribution over the actions.
- Let A be the set of all actions, e.g., $A = \{\text{"left"}, \text{"right"}, \text{"up"}\}$
- $\sum_a \pi(a|s;\theta) = 1$ (The reason why we use softmax activation).



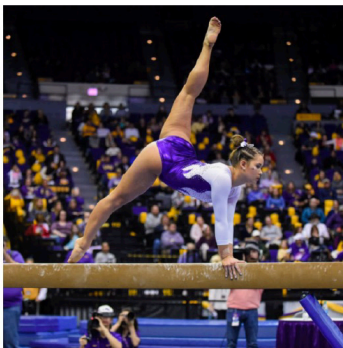
Value Network (Critic): $Q(s, a; \mathbf{w})$

- **Inputs:** state s (or with action a)
- **Output:** action-values of all actions

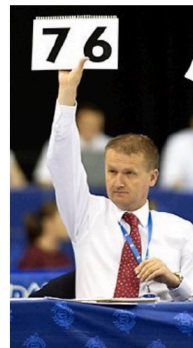


Actor-Critic Method

policy network (actor)



value network (critic)



Train the Networks

Definition: State-value function approximated using neural networks

- $V(s; \boldsymbol{\theta}, \boldsymbol{w}) = \sum_a \pi(a|s; \boldsymbol{\theta}) \cdot Q(s, a; \boldsymbol{w})$

Training: Update the parameters $\boldsymbol{\theta}$ and $\boldsymbol{w}$

- Update policy network $\pi(a|s; \boldsymbol{\theta})$ to increase the state-value $V(s; \boldsymbol{\theta}, \boldsymbol{w})$
 - Actor gradually performs better
 - Supervision is purely from the value network (critic)
- Update value network $Q(s, a; \boldsymbol{w})$ to better estimate the return
 - Critic's judgement becomes more accurate
 - Supervision is purely from the rewards

Train the Networks

Definition: State-value function approximated using neural networks

- $V(s; \boldsymbol{\theta}, \boldsymbol{w}) = \sum_a \pi(a|s; \boldsymbol{\theta}) \cdot Q(s, a; \boldsymbol{w})$

Training: Update the parameters $\boldsymbol{\theta}$ and $\boldsymbol{w}$

- 1 Observe the state s_t
- 2 Randomly sample action a_t according to $\pi(\cdot|s_t; \boldsymbol{\theta}_t)$
- 3 Perform a_t and observe new state s_{t+1} and reward r_t
- 4 Update $\boldsymbol{w}$ (in value network) using temporal difference (TD)
- 5 Update $\boldsymbol{\theta}$ (in policy network) using policy gradient

Update Value Network Q using TD Learning

- Compute $Q(s_t, a_t; \mathbf{w}_t)$ and $Q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$
- TD target: $y_t = r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$
- Loss: $L(\mathbf{w}) = \frac{1}{2}[Q(s_t, a_t; \mathbf{w}_t) - y_t]^2$
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}|_{\mathbf{w}=\mathbf{w}_t}$

Update Policy Network π using Policy Gradient

Definition: State-value function approximated using neural networks

- $V(s; \theta, w) = \sum_a \pi(a|s; \theta) \cdot Q(s, a; w)$

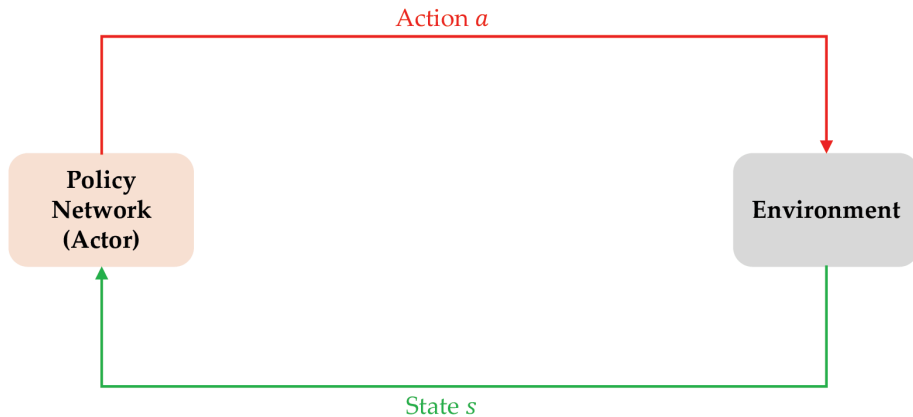
Policy Gradient: Derivative of $V(s; \theta, w)$ w.r.t. θ

- Let $\mathbf{g}(a, \theta) = \frac{\partial \log \pi(a|s; \theta)}{\partial \theta} \cdot Q(s_t, a; w)$, i.e., $\frac{\partial V(s; \theta, w)}{\partial \theta} = \mathbb{E}_a[\mathbf{g}(a, \theta)]$

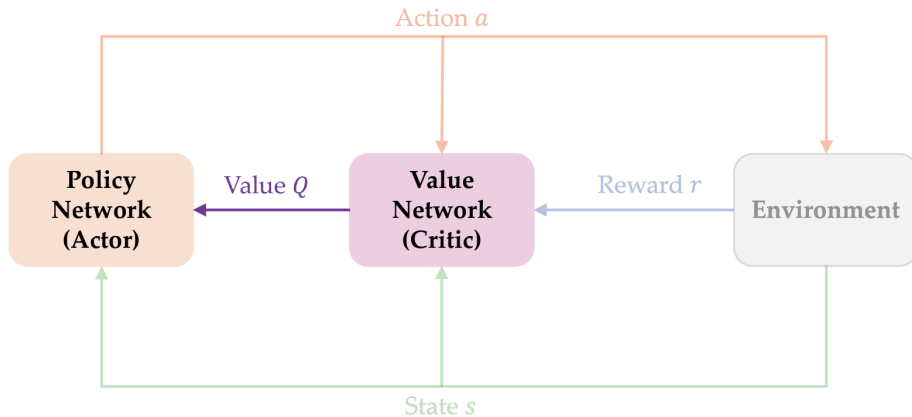
Algorithm: Update policy network using stochastic policy gradient

- Random sampling: $a \sim \pi(\cdot|s_t; \theta_t)$. ($\mathbf{g}(a; \theta)$ is unbiased)
- Stochastic gradient ascent: $\theta_{t+1} = \theta_t + \beta \cdot \mathbf{g}(a, \theta_t)$

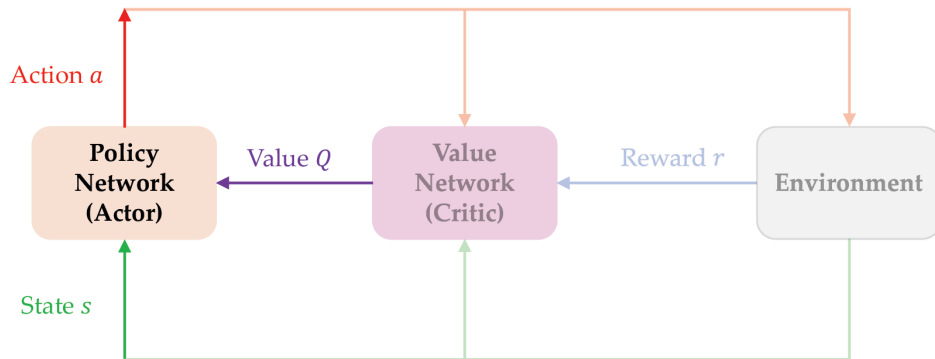
Actor-Critic Method



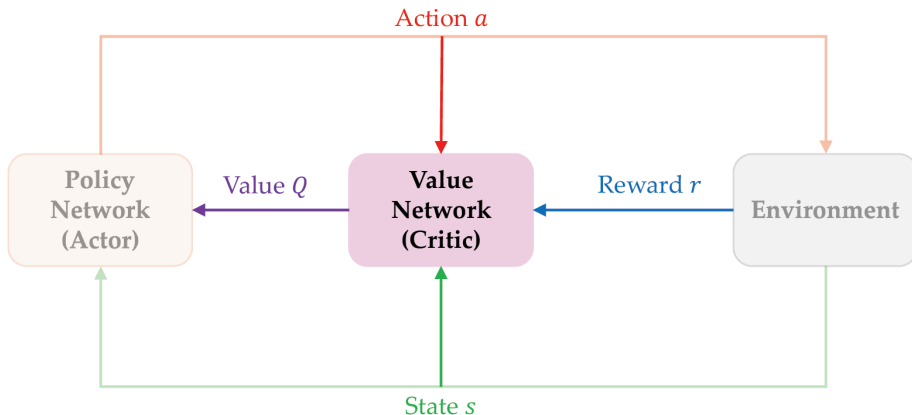
Actor-Critic Method



Actor-Critic Method



Actor-Critic Method



Summary of Algorithm

- 1 Observe state s_t and randomly sample $a_t \sim \pi(\cdot|s_t; \theta_t)$
- 2 Perform a_t ; then environment gives new state s_{t+1} and reward r_t
- 3 Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot|s_{t+1}; \theta_t)$ (Do not perform $\tilde{a}_{t+1}!$)
- 4 Evaluate value network: $Q_t = Q(s_t, a_t; w_t)$ and $Q_{t+1} = Q(s_{t+1}, \tilde{a}_{t+1}; w_t)$
- 5 Compute TD error: $\delta_t = Q_t - (r_t + \gamma \cdot Q_{t+1})$
- 6 Differentiate value network: $d_{w,t} = \frac{\partial Q(s_t, a_t; w)}{\partial w} \big|_{w=w_t}$
- 7 Update value network: $w_{t+1} = w_t - \alpha \cdot \delta_t \cdot d_{w,t}$
- 8 Differentiate policy network: $d_{\theta,t} = \frac{\partial \log \pi(a_t|s_t; \theta)}{\partial \theta} \big|_{\theta=\theta_t}$
- 9 Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot Q_t \cdot d_{\theta,t}$

Roles of Actor and Critic

During training

- Agent is controlled by policy network (actor): $a_t \sim \pi(\cdot|s_t; \theta)$
- Value network (critic) provides the actor with supervision

After training

- Agent is controlled by policy network (actor): $a_t \sim \pi(\cdot|s_t; \theta)$
- Value network (critic) will not be used

① Actor-Critic

② Advantage Actor-Critic (A2C)

③ Design Decisions

What about the Baseline for Actor-Critic

- True expected reward-to-go:

$$Q(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}} [r(s_{t'}, a_{t'}) | s_t, a_t]$$

- Policy gradient with baseline:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) (Q(s_{i,t}, a_{i,t}) - b)$$

- Baseline can be:

$$b = \frac{1}{N} Q(s_{i,t}, a_{i,t})$$

- Or:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi_{\theta}(a_t | s_t)} [Q(s_t, a_t)]$$

State & State-Action Value Function

- $Q_{\pi}(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}}[r(s_{t'}, a_{t'}) | s_t, a_t]$: total reward from taking a_t in s_t
- $V_{\pi}(s_t) = \mathbb{E}_{a_t \sim \pi_{\theta}(a_t | s_t)}[Q_{\pi}(s_t, a_t)]$: total reward from s_t
- $A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$: how much better a_t is (Advantage function)

$$\nabla_{\theta} \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \cdot A_{\pi}(s_t, a_t)$$

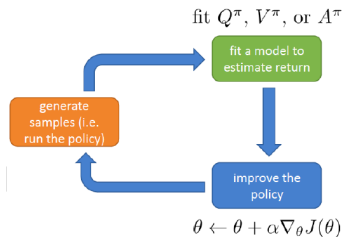
- $A_{\pi}(s_t, a_t)$: The better this estimation, the lower the variance

$$\nabla_{\theta} \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left(\sum_{t'=1}^T \cdot (r(s_{i,t}, a_{i,t}) - b) \right)$$

- $(r(s_{i,t}, a_{i,t}) - b)$: Single-sample estimation, unbiased but high variance

Value Function Fitting

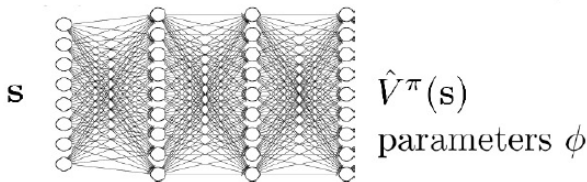
- Q: Fit what to what? Q_π, V_π, A_π ?



- $Q_\pi(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta} [r(s_{t'}, a_{t'}) | s_t, a_t]$
- $Q_\pi(s_t, a_t) = r(s_t, a_t) + \sum_{t'=t+1}^T \mathbb{E}_{\pi_\theta} [r(s_{t'}, a_{t'}) | s_t, a_t]$
- $Q_\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(s_{t+1} | s_t, a_t)} [V_\pi(s_{t+1})]$
- $Q_\pi(s_t, a_t) \approx r(s_t, a_t) + V_\pi(s_{t+1})$ single-sample estimate for single-step

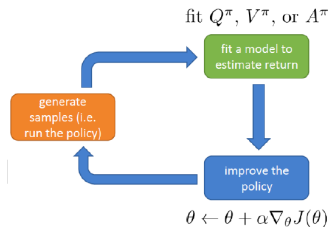
Value Function Fitting

- $Q_{\pi}(s_t, a_t) \approx r(s_t, a_t) + V_{\pi}(s_{t+1})$ single-sample estimate for single-step
- $A_{\pi}(s_t, a_t) \approx r(s_t, a_t) + V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$
- Lets just fit $V_{\pi}(s_t)$!



Policy Evaluation with MC

- $V_{\pi}(s_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}}[r(s_{t'}, a_{t'}) | s_t]$
- Monte Carlo policy evaluation (this is what policy gradient does)
- $V_{\pi}(s_t) \approx \frac{1}{N} \sum_i \sum_{t'=t}^T r(s_i, t', a_{i,t'})$



Function Approximation

Monte Carlo Evaluation

- $V_{\pi}(s_t) \approx \frac{1}{N} \sum_i \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'})$
- Training data: $\{(s_{i,t}, \sum_{t'=t}^T r(s_{t'}, a_{t'}) = y_{i,t})\}$
- Supervised regression: $L(\phi) = \frac{1}{2} \sum_i \|\hat{V}_{\phi}^{\pi}(s_i) - y_i\|^2$ **Q: Can we do better?**
- Ideal target
$$y_{i,t} = \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}}[r(s_{t'}, a_{t'}) | s_t] \approx r(s_{i,t}, a_{i,t}) + V_{\pi}(s_{i,t+1}) \approx r(s_{i,t}, a_{i,t}) + \hat{V}_{\phi}^{\pi}(s_{i,t+1})$$
Directly use previous fitted value function (“bootstrapped” estimate)
- Training data: $\{(s_{i,t}, r(s_{i,t}, a_{i,t}) + \hat{V}_{\phi}^{\pi}(s_{i,t+1}))\} = y_{i,t}$
- Supervised regression: $L(\phi) = \frac{1}{2} \sum_i \|\hat{V}_{\phi}^{\pi}(s_i) - y_i\|^2$

Advantage Actor-Critic (A2C) Algorithm

Batch Actor-Critic

1. Sample τ_i from $\pi_{\theta}(a_t|s_t)$ (run the policy)
2. Fit $\hat{V}_{\pi}(s)$ to sampled reward sums
3. Evaluate $\hat{A}_{\pi}(s_i, a_i) = r(s_i, a_i) + \hat{V}_{\pi}(s'_i) - \hat{V}_{\pi}(s_i)$
4. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i, s_i) \hat{A}_{\pi}(s_i, a_i)$
5. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$

(LOOP) GO TO STEP1

Advantage Actor-Critic (A2C) Algorithm

Batch Actor-Critic

1. Sample $\{s_i, a_i\}$ from $\pi_{\theta}(a_t|s_t)$
 2. Fit $\hat{V}_{\pi}(s)$ to sampled returns
 3. Evaluate $\hat{A}_{\pi}(s_i, a_i) = r(s_i, a_i) + \gamma \cdot \hat{V}_{\pi}(s'_i) - \hat{V}_{\pi}(s_i)$
 4. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i|s_i) \hat{A}_{\pi}(s_i, a_i)$
 5. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

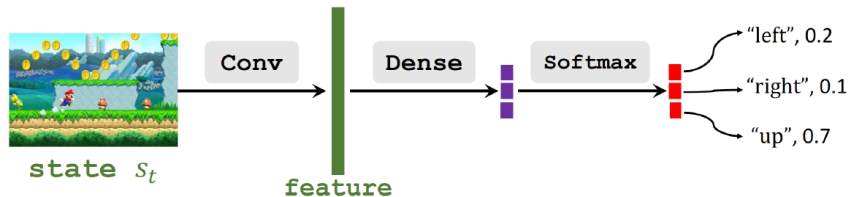
Online Actor-Critic

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r)
 2. Update $\hat{V}_{\pi}(s)$ using target $r + \gamma \cdot \hat{V}_{\pi}(s')$
 3. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
 4. Calculate $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(a, s) \hat{A}_{\pi}(s, a)$
 5. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

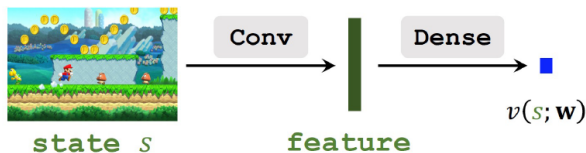
- ① Actor-Critic
- ② Advantage Actor-Critic (A2C)
- ③ Design Decisions

Policy and Value Network

Policy Network

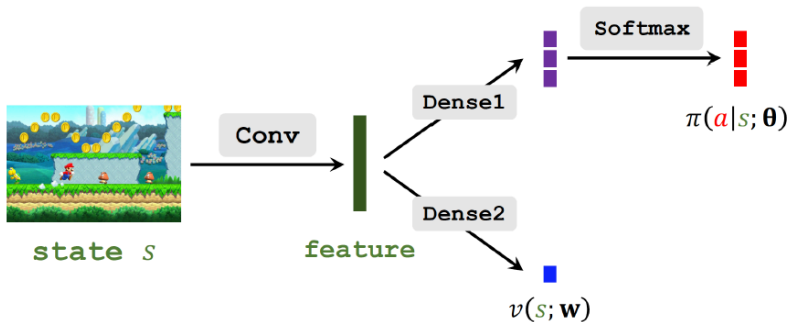


Value Network



Policy and Value Network

Parameter Sharing



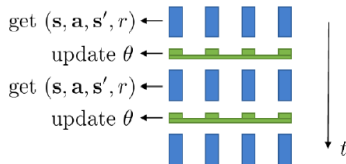
Online Actor-Critic In Practice

Online Actor-Critic

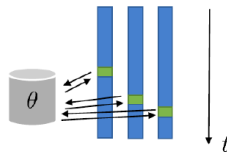
1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r)
2. Update $\hat{V}_{\pi}(s)$ using target $r + \gamma \cdot \hat{V}_{\pi}(s')$
3. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
4. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a, s) \hat{A}_{\pi}(s, a)$
5. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$

(LOOP) GO TO STEP1

synchronized parallel actor-critic



asynchronous parallel actor-critic



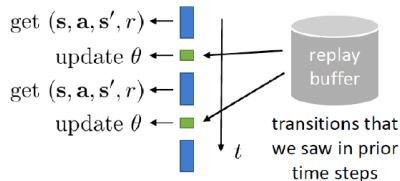
Can we remove the on-policy assumption?

Online Actor-Critic

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r)
2. Update $\hat{V}_{\pi}(s)$ using target $r + \gamma \cdot \hat{V}_{\pi}(s')$
3. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
4. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a|s) \hat{A}_{\pi}(s, a)$
5. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$

(LOOP) GO TO STEP1

Off-policy actor-critic



Can we remove the on-policy assumption?

Online Actor-Critic with Replay Buffer

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r) , store in replay buffer B
 2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer B
 3. Update $\hat{V}_{\pi}(s)$ using target $y_i = r_i + \gamma \cdot \hat{V}_{\pi}(s'_i)$ for each s_i
 4. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
 5. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a|s) \hat{A}_{\pi}(s, a)$
 6. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

Q: The algorithm is broken! Can you spot the problem?

Can we remove the on-policy assumption?

Online Actor-Critic with Replay Buffer

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r) , store in replay buffer B
 2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer B
 3. Update $\hat{V}_{\pi}(s)$ using target $y_i = r_i + \gamma \cdot \hat{V}_{\pi}(s'_i)$ for each s_i
 4. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
 5. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a|s) \hat{A}_{\pi}(s, a)$
 6. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

Q: The algorithm is broken! Can you spot the problem?

STEP 3: $y_i = r_i + \gamma \cdot \hat{V}_{\pi}(s'_i)$ not the right target value

STEP 5: $\nabla_{\theta} \log \pi_{\theta}(a|s)$ not the action π_{θ} would have taken!

Online Actor-Critic In Practice

Online Actor-Critic with Replay Buffer

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r) , store in replay buffer B
2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer B
3. Update $\hat{V}_{\pi}(s)$ using target $y_i = r_i + \gamma \cdot \hat{V}_{\pi}(s'_i)$ for each s_i
4. Evaluate $\hat{A}_{\pi}(s, a) = r(s, a) + \gamma \cdot \hat{V}_{\pi}(s') - \hat{V}_{\pi}(s)$
5. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a|s) \hat{A}_{\pi}(s, a)$
6. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$

(LOOP) GO TO STEP1

STEP 3: Update $\hat{Q}_{\pi}$ using targets $y_i = r_i + \gamma \cdot \hat{V}_{\pi}(s'_i)$ for each s_i, a_i

$= r_i + \gamma \cdot \hat{Q}_{\pi}(s'_i, a'_i)$

not from replay buffer B , $a'_i \sim \pi_{\theta}(a'_i|s'_i)$

Online Actor-Critic In Practice

Online Actor-Critic with Replay Buffer

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r) , store in replay buffer B
 2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer B
 3. Update $\hat{Q}_{\pi}(s)$ using target $y_i = r_i + \gamma \cdot \hat{Q}_{\pi}(s'_i, a'_i)$ for each s_i, a_i
 4. Evaluate $\hat{A}_{\pi}(s, a) = \hat{Q}_{\pi}(s_i, a_i) - \hat{V}_{\pi}(s)$
 5. Calculate $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(a|s) \hat{A}_{\pi}(s, a)$
 6. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

STEP 5. Use the same trick, but this time for a_i rather than a'_i
Sample $a_i^{\pi} \sim \pi_{\theta}(a|s_i)$, not from the replay buffer B

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i^{\pi} | s_i) \hat{A}_{\pi}(s_i, a_i^{\pi})$$

Online Actor-Critic In Practice (with Replay Buffer)

1. Take action $a \sim \pi_{\theta}(a|s)$, get (s, a, s', r) , store in replay buffer B
 2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer B
 3. Update $\hat{Q}_{\pi}(s)$ using target $y_i = r_i + \gamma \cdot \hat{Q}_{\pi}(s'_i, a'_i)$ for each s_i, a_i
 4. Evaluate $\hat{A}_{\pi}(s, a) = \hat{Q}_{\pi}(s_i, a_i) - \hat{V}_{\pi}(s)$
 5. Calculate $\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i^{\pi} | s_i) \hat{A}_{\pi}(s_i, a_i^{\pi})$, where $a_i^{\pi} \sim \pi_{\theta}(a | s_i)$
 6. Update $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$
- (LOOP) GO TO STEP1

Q: Is there any remaining problem?

s_i didn't come from $p_{\theta}(s)$

Nothing we can do here, just accept it

Intuition: We want an optimal policy on $p_{\theta}(s)$, but we get an optimal policy on a **broader** distribution

Extra Reading Materials

- Chapter 13 on Policy Gradient Methods, Reinforcement Learning: An Introduction, Sutton & Barto
- Asynchronous Methods for Deep Reinforcement Learning. In ICML, 2016.
- Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In ICML, 2018.